Meetings were online every Wednesday at around 10:30am for sem 1.
The meetings were moved to Wednesday 4:30pm during the inter-sem break.
Then finally for sem 2 the meetings were moved to Thursday 4:30pm.

04/03/26

- Met with supervisor and co-supervisor
- Initialization of project and establishing communication (microsoft teams)

11/03/26

- Explored Claude via its CLI and implemented an simple agentic workflow to explore Claude's ADK
- Discussed future tools and some ideas of how to implement the system

18/03/26

- Played around with GitNexus - a tool for mapping out the dependencies between various files in the system
- Discussed its feasibility and how it can be integrated into the system

25/03/26

- Discussed literature review
- Discussed Claude plans and budget as $250 will not be able to cover the API usage
- Settled on used free gemini models and google's ADK

01/04/26

- Progress update on literature review
- Started implementing a prototype of the system with a main orchestrator agent, and repository analysis, planning and execution subagents.
- We discussed Gemini's limited context and token usage as it was severely limiting our testing capabilities

No meetings during the 2 week break.

30/04/26

- Continued implementing the prototype system with Gemini and discussed other possible AI usage while staying within budget
- Ideas about using OpenClaw and a Claude monthly subscription was explored but dropped due to rule violations
- In the end we settled on using Claude monthly subscription to do initial testing and development to stay within budget
- We plan to move to using an API key and moving to ADK as the main bulk of the work is the prompting (agent prompts + skill development)

06/05/26

- Discussed Claude's memory and context, especially when dealing with larger codebases
- In the end we decided to use OpenAI's codex models as they support a larger context window (400k tokens compared to Claude's 200k) and higher token availability
- Its coding capabilities are also comparable and thus we planned on switching to OpenAI once our current Claude subscription was over

13/05/26

- Discussed potential evaluation metrics for our tool such as passing tests and other static analysis methods
- Went over results from experiments with smaller repos and showed that functionality was intact after small refactors.

20/05/26

- Discussed context and memory limitations
- Upgrading libraries should be fine, but full architecture or framework migrations will prove to be extremely challenging due to the sheer amount of information the AI needs to handle
- Discussed the feasibility of implementing a test generator agent
- Cosupervisor confirmed this was a good starting point as its context will be isolated from the planning agent, removing bias (as AI can sometimes create or adjust tests so that it passes, even though the functionality tested was incorrect)

27/05/26

- Went over the testing agent and that it generated new test cases and the new and old test cases continued to pass after the refactor
- A new repo from the Library's digital team was went over as the next repository to test the dependency upgrades.

03/06/26

- Jinhao and Dhruv were busy this week so they could not get much work done for the meeting.
- The meeting was cancelled this week.

10/06/26

- Migrated our tool over from Claude CLI to instead use Codex CLI as recommended by Mengze earlier on in the project.
- Went over issues we faced in regards to our tool's token consumption: the full five hour window was used up by the refactor conversation.

- Jinhao noted issues with build issues that he had to provide another prompt to fix (not ideal since we want it to be as autonomous as possible)
- Made the scope of the project for the mid year report to only be library upgrades and to then increase it after it.
- Went over more evaluation metrics: comparing refactor time to AI estimated manual human refactors, comparing it with other library upgrade tools available online
- Asked how much repos to analyze would be satisfactory for the report (4 should be fine with good analysis)
- Brief description was gone over the report; tool description, results, analysis of results, comparison to other tools.
- The task for next week is to finish the tool so it has no issues and then start to get good results for the report.

17/06/26

- Went through our progress from last week
- No improvements to the tool found while testing
- Struggle to find the balance between having a tool that correctly upgraded all the dependencies found in the repo and didn't use up a lot of tokens
- Tools frequently used up the whole 5 hour codex window so we have limited test runs per week due to codex windows.
- According to GPT, one of our runs that upgraded all the dependencies correctly would have cost around $7.50 USD using the codex api instead of our cli which is our final goal.
    - Way too expensive for our current plan so we need to limit the amount of tokens our tool uses.
- Dhruv found some extra metrics and two different dependency upgrade tools (dependabot, rennovate, and copilot) that can be used in our mid-year report to measure how effective our tool is.
- Professor Sun said that our tool being finished is more important than our mid-year report being perfect
    - Can just use failed runs for our results and focus more on improving the tool
- We established that the validation agent was doing too much work as it was going through all the changes made by the sub agents it was validating which was burning tokens.

Metrics:
Upgraded dependencies / Outdated dependencies
Successful upgrades / Attempted upgrades
Build passes after upgrade?
Runtime - compare with human manual estimates
Token usage / context window with estimated costs
Manual Intervention counts
Rollback frequency

24/06/26 (Meeting was cancelled as Professor Sun could not make it)

- Dhruv managed to get the tool working to improve the frontend folder of the outdated repo

        - Changed how the validation worked for the tool
            - Instead of running after every subagent, it instead runs at the end and checks if the overall changes to the repo matches the change plan report.
        - Reduced the amount of tests created as multiple tests were created per changed file no matter how small the change was
        - The tool also asked the user how to run / compile the application
            - This would be run by the executor agent to ensure that the code changed still compiles
                - Executor agent gets 3 attempts per provided command to fix an errors found
                    - Matches with how other dependency upgrade tools work as they make changes and fix any errors
- Didn't manage to run the tool on the admin code yet because Dhruv is still setting it up on his end.
    - Did still refactor the admin code but didnt run it so no clue if it even works
    - Will need to rerun the tool with providing it build commands for the admin code.
- Only used 46% of the context window and GPT reports that if a codex API was used instead it would have cost $2.56 USD which is a huge drop from earlier.
- Main issue is that the testing agent was reduced a lot so only two detailed tests were developed since it thought that was necessary based on the change plan
- Jinhao improved upon tooling usage and regex querying since poor queries can result in massive outputs, eating up large amounts of context unnecessarily
- Jinhao managed to get the tool working on frontend with minor error in backend using approx. 1 and a half sessions. (This is down from the initial 2 sessions of our base system)
- Jinhao also experimented more with memory compaction and progressive loading to reduce context size. (This needs further testing)
    - This can help improve memory usage and performance (we observed cases where the agents talk to each other quite frequently, which causes token usage to compound.)
    - Proactive compaction checkpoints were added so long runs preserve context before approval gates and after validation artifacts are written

1/07/26

- Dhruv ran the admin code and had errors which indicated that the upgrade didn't work
- Dhruv provided the command to include the admin build command and had no errors when compiling the code but when running the repo's code, the frontend code and admin page couldn't find the website when Dhruv went to the localhost pages.
    - The validator agent also failed the upgrade due to some error that happened during the upgrade (dk what it is, I would have to get the logs for it)
    - 60% was used

- Dhruv modified his tool to also curl requests to the localhost's websites to check that they were live.
- 96% of the 5 hour context window was used.
- It ended up working as both the API and the frontend code was working but the problem was that the chapman.js module and commands didn't work and interface also didn't load properly (blank page).

For Mengze:

- Can we provide more commands instead of only build commands for the tool to test with to ensure functionality is maintained. (i.e. chapman commands)
- This will burn way more tokens
    - I would think that the previous run would be around $5 (forgot to save token usage)
- My issue with that is that I really don't know how useful our tool would be to use if the user has to provide a lot of commands for their code in order to set up their repo.
- If we don't do that, how would we make this tool actually work?
    - There are so many changes being made, it's hard to tell that the changes do not affect code functionality.
    - Testing agents do not generate high quality tests that test the functionality of the repo that well.

8/07/26 (Meeting was cancelled because Professor Sun made a mistake so it actually happened on the 14th)

- Dhruv was unable to work on the project this week due to a trip but had communicated this in advance.
- Mengze said that he wanted to see some results of our tool refactoring the tv-radio repository.

Dhruv's side of the results:
First run: didn't provide any build commands and told it to just refactor any outdated dependencies in the codebase.
Used 49% of the context window, and worked for around ~47 minutes.
Token usage: total=169,717 input=155,892 (+ 945,280 cached) output=13,825 (reasoning 2,181). According to GPT, this would cost ~1.67 USD if using the OpenAI API.
According to Claude, 21 out of 21 dependencies were correctly upgraded for frontend and in total 49 out of 51 dependencies were successfully upgraded. Pace and optimist dependencies were not upgraded as they are apparently abandoned and have no latest versions.
Pretty sure the frontend worked for this run but dont think admin code worked.

Second run: Changed the tool to try and build the tool and fix any errors found. Only ran on the front end to test it.
Used 46% of the context window, and worked for ~33 minutes.
Forgot to save the token usage but when I put it into gpt, the api would have cost $2.54 USD.

Dependencies changed was the same as before for frontend but gc-stats and uuid were removed. Gc-stats was not referenced but uuid was still referenced showing an error.

Third run: Provided build commands for both frontend and admin code.
Used 60% of the context window and ran for 34 minutes.
Token usage: total=183,862 input=165,654 (+ 1,564,544 cached) output=18,208 (reasoning 2,033). API estimated cost = $2.16 USD
Had an issue where it got rejected by the validator before changing admin code due to git branch issues (one off issue).
Frontend dependencies faced the same issues as the previous run.
Both the frontend and backend ran and compiled with no issues but when you went to the websites there was no content being displayed.

Fourth run: Provided the url links as well and changed the tool to send curl requests and check that there were responses for the websites.
Used 94% of the context window. Token usage was not saved so GPT estimated a cost of $0.90-$1.40 for this run.
Ran for ~59m. Everything worked and compiled but some Chapman commands which are required for indexing and searching failed.
Dependencies changed were the same as the first run.

Jin's Side of results
First run:

- Implemented early compaction
- Batching changes so we can fascilitate spawning more agents for different batches in order to reduce token compounding consumption

2.4 Sessions ~$12, ~15m tokens, all libraries got updated correctly, but missed updating the usage in a file, causing runtime error. (running code in metrics.js resulted in an error due to incorrect use of the influx library.

Second run:

- Also started writing digests for file inspections and dropping raw output as to not clog up context
- Made compaction more aggressive
    - Every 10 tool calls within a single stage, regardless of context size: write all pending digests, then check approximate context usage. If usage exceeds 35%, compact before the next tool call. If usage is below 35%, continue but record the check in `token-accounting.json`.
- Full file read is only allowed if there is a strong reason to do so (e.g. high risk)
- Added artifact escalation as well (read as needed)

2.5 Sessions ~$13, ~17m tokens, all libraries updated, frontend seemed to be working, however upon trying to create indices using chapman commands, errors were found.

Third run:

- Previously, we always prioritized correctness while trying to cutdown token usage
    - It seems this approach is not quite possible
- We'll aim to mimmick real life software engineers more closely and perform test driven development and use iteration
    - We still analyze, plan and upgrade as usual, but instead of trying to one shot it, we aim more for token efficiency and rely on post upgrade testing (compiling and running the application)

2.93 Sessions ~$18, ~17m tokens, all libraries got updated correctly, upgrade completely failed, agent kept running into issues and got stuck in loop while performing upgrade, resulting in multiple restarts of a batch of upgrades. Both frontend and backend failed in the end

4th Run:

- Use a progressive tooling approach output limits
- Changes to analysis scoping (previously analyzes everything, not necessary for some upgrade e.g library)

0.52 Sessions, ~$3, ~4m tokens, upgrade halted due to an error detected in the plan during the execution phase where performing validation after the first batch of changes, it encountered an error in the metrics.js file (it missed the metrics.js file despite it utilizing the influx library, which is one of the libraries that needed the upgrade, note that this particular run uses the new GPT 5.6 Sol on low reasoning and could have been the reason why less session and tokens were used.)

5th Run (GPT 5.6 Terra - Medium Reading):

- Added more compaction checkpoints in orchestrator
    - Orchestrator is kept alive the longest, and is the single biggest offender when it comes to compounding token consumption and carries many unnecessary information (tool calls, subagent completion polling, preflight kept)
- There are also other schema bugs/mismatches causing agents to be confused and waste many tokens trying to fix issues

0.12 sessions ~$6.7, ~1m tokens, ~batch 1 finished but upgrade halted due to error detected in planned test cases. The planned test cases assumed old mongodb version API usage. It seems the planning of test cases goes into too much detail (it probably should only define data input and outputs, also should allow for recovery rather than immediate failure and halting the upgrade pipeline – i.e. its too strict)

For next week:
Provide a chapman command and see if it compiles correctly after that.
Get started on the mid-year report.

18/07/26

- Consolidated results and writing mid year report
- Began preparing slides and practicing presentation

- From this point we will continue on with Dhruv's implementation and trying to improve upon that

Changes made to tool:

Run 1: Added specialized prompting dependency/library upgrade

- Validation failed initially as Express 5 rejects /media/access/* in tv-radio-frontend/usage-monitoring.js, resulting in upgrade halt
- This file was missed in the change plans
- Agent provided next steps: add usage-monitoring.js to the upgrade plan
- Agent was allowed to continue
- archiver@8.0.0 fails to load on Node 22 because its ESM code imports a named export from CommonJS readable-stream@4.7.0. A known compatible target is archiver@7.0.1, but changing the explicitly approved 8.0.0 target would violate the plan, so no commit can be created.
- Rejected again: It requires randomUUID in a file where the approved change intentionally removes unused UUID code. It globally forbids version 8.0.0 in the Export lockfile, matching an unrelated emoji-regex dependency;

Token usage: total=542,848 input=515,991 (+ 14,764,800 cached) output=26,857
Test cases: 2
Duration: ~92mins

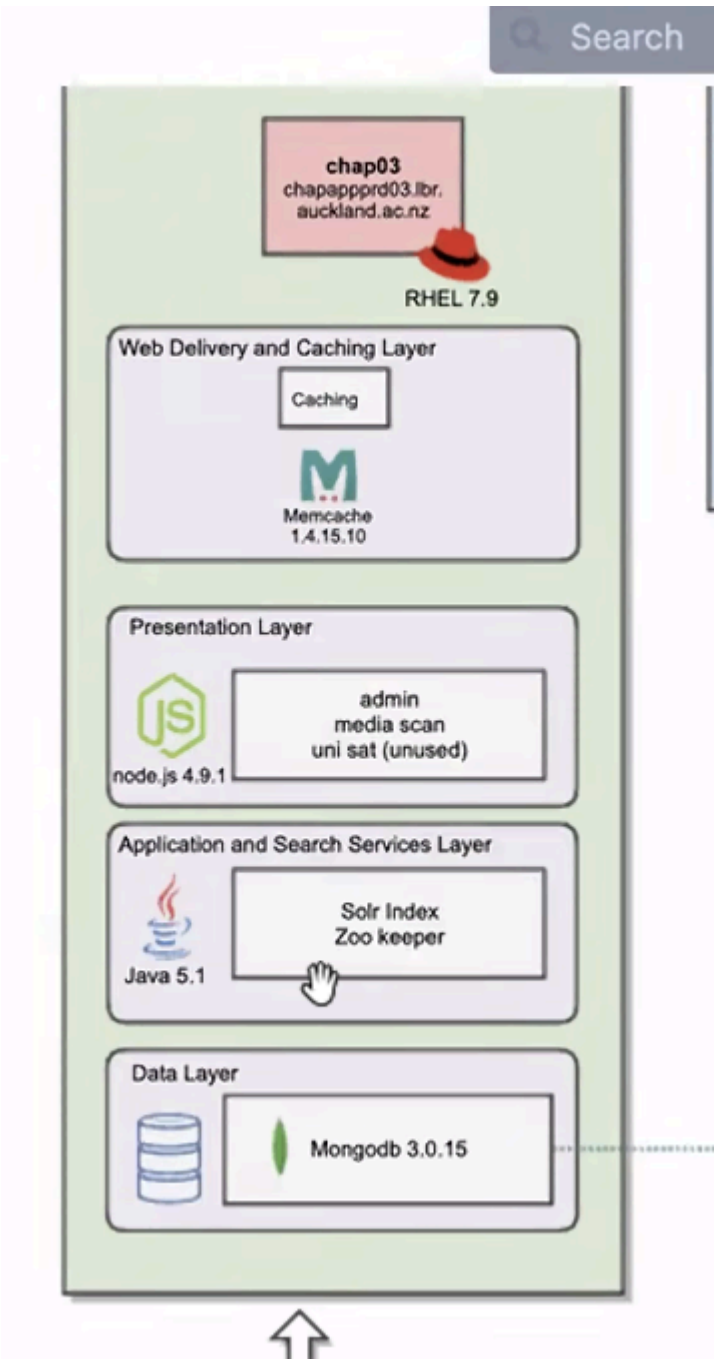
Notes:

- **Frontend appears operational while backend runs but going to the webpage displays nothing with errors in the console**
- **Frontend display errors**: {"level":"error","message":"[Data/Homepage] generating list of most populous genres failed due to: Error: Solr response didn't contain a valid status code in response header (400)."}
- However, this is more so due to the Solr setup
- Also **admin's solrconfig.xml** was not updated and remained at version 4.9 despite SolR version being changed to 8.11

30/07/26

- Talked to Yin Yin (a staff of the University Library system)
- Showed her our most successful upgrade (frontend mostly working with some errors in admin + chapman commands hanging)
- She explained that its likely due to not having proper data in our solr and/or mongodb database
- Also said that "Unisat" functionality can be ignored since its a retired part of the system

https://collections.library.auckland.ac.nz/tv-radio/
https://solr.apache.org/guide/solr/latest/getting-started/solr-indexing.html



6/08/2026

- Discussed Playwright MCP with supervisor and co-supervisor as a potential tool to drastically improve the agent's post upgrade error analysis and debugging capabilities
- Started experimenting with Playwright which allows our agent interact with web pages rather than just running simple CURL requests
- Playwright sends back structured snapshots which provide much more valuable information and even allows us to access components such as the console, giving the agent a greater insight into errors that aren't logged on the terminal it runs the application on

- Dhruv managed to get a run with no errors in frontend and admin component after a few manual fixes
- Since we do not have access the full TV-Radio local setup, we will be passing this onto our co-supervisor for further testing

13/08/2026

- Jinhao found some issues with the api routes being used in the refactored system as there was an extra / in the routes e.g. /api//media-scan so the backend was not responding to the frontend in the admin interface.
- Dhruv changed the validator agent as he noticed that it only checked the first page it landed on so he made it actually check that all the pages in the system worked as intended
- Last week's refactored system had issues on Jinhao's end but this new refactored system worked with no major issues.
- This upgraded repo replaced the old one and we shared it to Mengze for further testing using the library system's testing suite
- We provided files that we used when setting up the refactored codebases on our ends to Mengze and the library team.

20/08/2026

- Mengze said he had issues with the testing suite as he was missing permissions so he passed it onto Yin Yin (Library services team)
- Yin Yin said she would be busy as the IT team was working on the systems and she would be out of the office for a week so we would have to wait until September to hear any progress on the upgraded system
- We had issues with our validator agent as it wasn't really using a defined rubric to mark how correct our upgraded repo was.
- Jin worked on it and defined a detailed rubric for the validator to follow.
- Initially it didn't seem accurate as a failing upgrade actually passed the validator with a score of 80%
- Jin tweaked the validator agent some more and got some more realistic results for failing upgrades (35%).

27/08/2026 (cancelled as Dhruv and Jin are busy with tests this week)

03/09/2026

- Mengze provided access to another repository from the library team via BitBucket
- Jin noticed that there were some dependencies that we didn't have access to
- Jin asked Yin Yin about it but she said that since we are students we don't have access to the resources
- Mengze provided some extra resources to help us out with this issue
- Professor Sun recommends that we try running our tool on other legacy systems to evaluate it for our final report
- Professor Sun also said to change the tool to have a prompting agent so that the user doesn't have to make a detailed prompt for each repo it creates. It should contain information about the new environment they want the repo to upgrade to

- Professor Sun also told us that we should focus on improving the tool and getting more results in September and focus on writing the report in October

10/09/26

- Geohub repo is burning through tokens quickly
- This affects us as the one Geohub refactoring run is using up the whole weekly limit
- Will be looking into new legacy systems (from small to medium to large size with different languages potentially) to use when the weekly limit is restored.
- Could just scrap the GeoHub run and focus on other smaller legacy system upgrades to get more results.

Java

- Bad Fruit from Peel & Slice (practice) - 29
    - This was a very simple repo, no user interface and upgrade passed
- ChartSmart (practice) - 225

Javascript

- https://github.com/codemirror/codemirror5 - 582

Npm install:

- npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache if you want a good and tested way to coalesce async requests by a key value, which is much more comprehensive and powerful.
- npm warn deprecated rimraf@2.7.1: Rimraf versions prior to v4 are no longer supported
- npm warn deprecated glob@7.2.3: Old versions of glob are not supported, and contain widely publicized security vulnerabilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exorbitant rates) by contacting i@izs.me
- npm warn deprecated sourcemap-codec@1.4.8: Please use @jridgewell/sourcemap-codec instead
- npm warn deprecated acorn-dynamic-import@4.0.0: This is probably built in to whatever tool you're using. If you still need it... idk
- npm warn deprecated puppeteer@1.20.0: < 24.15.0 is no longer supported

Didnt work because tests were failing.

Both assert on exact pixel geometry (cm.cursorCoords(), a hard-coded 5px scroll tolerance) and bidi cursor-movement direction, which depend entirely on Chromium's own text-layout/UBA engine — not on any CodeMirror, Rollup, or Puppeteer API. Tellingly, test.js already has a comment a few lines below the bidi test helper:

// We don't correctly implement L1 UBA

// See https://bugzilla.mozilla.org/show_bug.cgi?id=1331501

// and https://bugs.chromium.org/p/chromium/issues/detail?id=673405

i.e. the suite was already known to be sensitive to Chromium's bidi implementation quirks back when it was written against Puppeteer 1.20's bundled Chromium (~v69, 2018). Puppeteer 25 bundles a Chromium roughly seven years newer, whose bidi/layout engine has genuinely changed. The two "repairs" the executor applied (Vim keymap transform, 404 handling in test/run.js) are nowhere near this code — they couldn't have fixed these failures, and the executor correctly figured that out on retry ("deterministic Chromium 152 layout/bidi assertion differences, not... API migration issues").

- https://github.com/monarch-initiative/monarch-legacy - 589
    - Token usage: total=1,450,532 input=1,386,132 (+ 37,883,520 cached) output=64,400 (reasoning 20,577)
    - Upgrade paused a few times but after telling it to just keep fixing issues, it eventually managed to fix itself
    - But we have the following error on the webpage itself: Cannot read properties of undefined (reading 'length'). No errors found in console or terminal
    - The errors on the page wasn't something the upgrade system caused
        - It has external web dependencies that no longer exist and were completely out of scope for the upgrade.
        - The upgrade system simply preserved the functionality and original code behavior. Thus the upgrade is still deemed successful

Python/JS

- https://github.com/Rhizome-Conifer/conifer - 834

17/09/26

- Went over the results obtained from three of the 5 found outdated repos
- Dhruv brought up the question as to whether we use the old test cases that are outdated that come with the repo.
- Professor Sun recommended upgrading those tests as well but since there is a lack of time to implement these changes to the testing agent we decided to analyze the failing tests and note them.
- For next week we planned to upgrade the final two outdated repos we found and to get results based on the metrics Mengze provided.

24/09/26

- Dhruv ran the upgrade on the Conifer repo with it passing both the tooling validation and manual validation
- Jin started rerunning the upgrade on the previous repos without the support of our upgrade system to gather baseline results
    - **Coderetreat**: Failed as only a partial upgrade was performed with almost all build.gradle files still retaining pre upgrade versions, meaning the LLM completely missed these files

    - **Monarch legacy**: Failed the upgrade since the webpack.config.js proxy configuration wasn't fully updated for a transitive dependency change. webpack-dev-server was correctly bumped to ^5.1.0, and the obvious v5 devServer option renames were done properly. But webpack-dev-server@5 pulls in http-proxy-middleware@3, which tightened validation on devServer.proxy[].context: the array must now be all glob patterns or all plain string paths — no mixing**.**
- Professor Sun asked us to provide us with our proposed report structure so that he can provide feedback for it next week

1/10/26 (cancelled due to systems week)

8/10/26

Since the project poster was due Sunday, we were discussing the poster and what to improve on the poster.

- Jinhao and Dhruv had already made a draft of the poster
- Professor Sun requested some changes to the poster in terms of information
- Recommendations
    - remove iterations of the tool and just have the final tool description
    - Have a more detailed architecture diagram of the tool.
    - Remove references
    - Remove paragraphs and add bullet points for quick reads
    - Add research questions to relate data to.
- Professor Sun also reviewed our proposed report structure.
    - We were told to remove the sections regarding the iteration of the tool from our mid year tool and just focus on the finalised version of the tool.
    - We were also told to include sections for discussing the evaluation metrics and have a discussion section to talk about the results and what that meant for the project.

15/10/26

- Dhruv and Jin spent last week working on their own individual reports and are working to finish it before Sunday.
- We discussed in the meeting to make a readme file to instruct others how to use the tool for the compendium assignment.
- Professor Sun told us about the display day presentation and that we would have to present for 20 minutes.
    - Recommended us to have a live demonstration of the tool or record a video.
    - We decided to record a video of our tool working
    - Dhruv and Jinhao said that they would work on the script for their poster day after submitting the final report.
- Discussed other things related to the final report like AI usage, how Professor Sun could sign the report after we were done when he is busy, and what he expects to see in the report.

- Professor Sun also recommended us to make some progress on the geohub repository if we have time for professionalism as the Library team provided us the repo for us to work on.
    - Dhruv and Jinhao said that we would work on the project if we get time but we don't think our tool could do that much with that repository since the repo is 10x the size of the repositories we were dealing with and required a lot of time and weekly limits of our tool already and it didn't seem close to being done.